

Celebal Summer Internship 2026 – SQL Week 2

Project: SQL Data Analysis

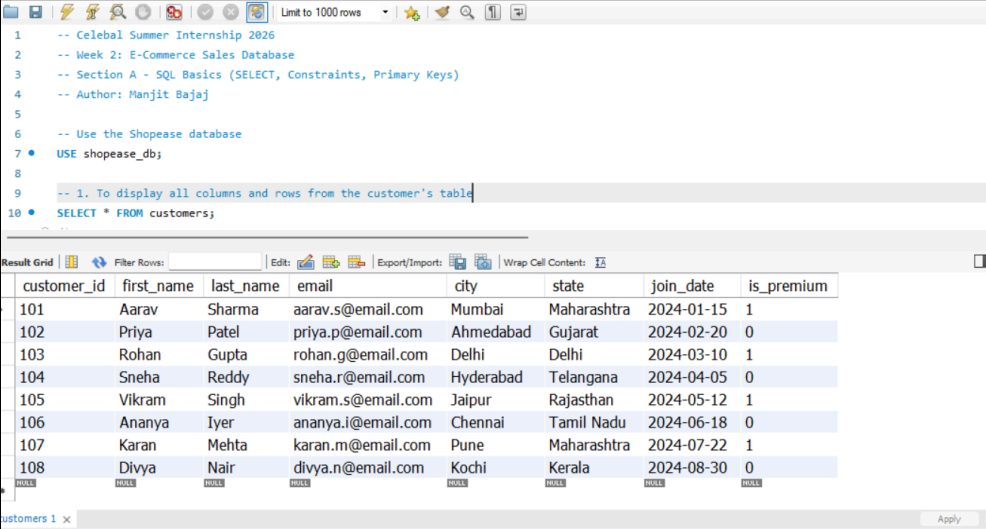
Internship: Celebal Summer Internship 2026 – SQL Week 2

Author: Manjit_Bajaj

Purpose

This document contains screenshots of the SQL queries executed during the analysis along with their corresponding outputs. These screenshots serve as evidence of successful query execution and support the findings presented in the project.

Section A - SQL Basics (SELECT, Constraints, Primary Keys)



The screenshot shows a SQL IDE window with a script editor and a result grid. The script editor contains the following SQL code:

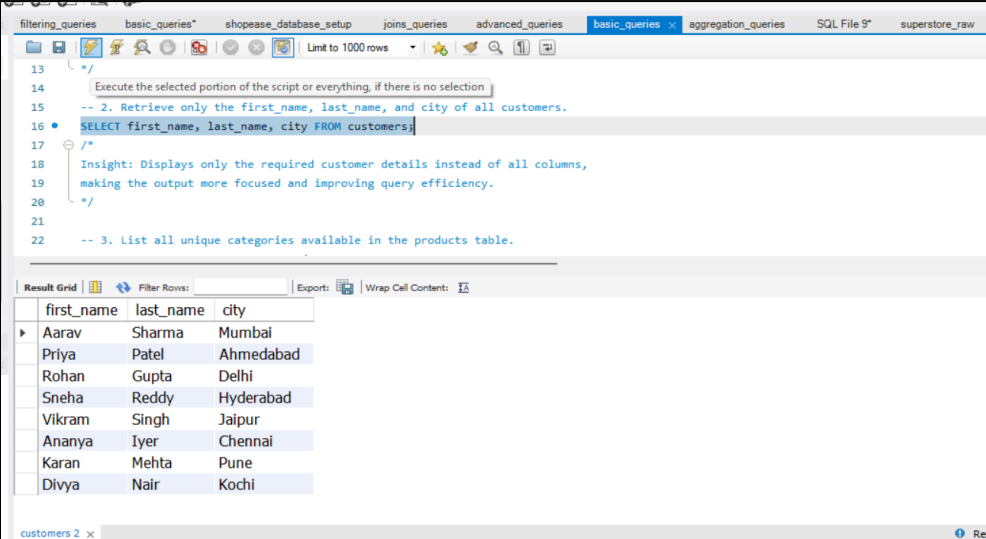
```
-- Celebal Summer Internship 2026
-- Week 2: E-Commerce Sales Database
-- Section A - SQL Basics (SELECT, Constraints, Primary Keys)
-- Author: Manjit Bajaj

-- Use the Shopease database
USE Shopease_db;

-- 1. To display all columns and rows from the customer's table
SELECT * FROM customers;
```

The result grid displays the output of the query, showing 8 columns: customer_id, first_name, last_name, email, city, state, join_date, and is_premium. The data is as follows:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0



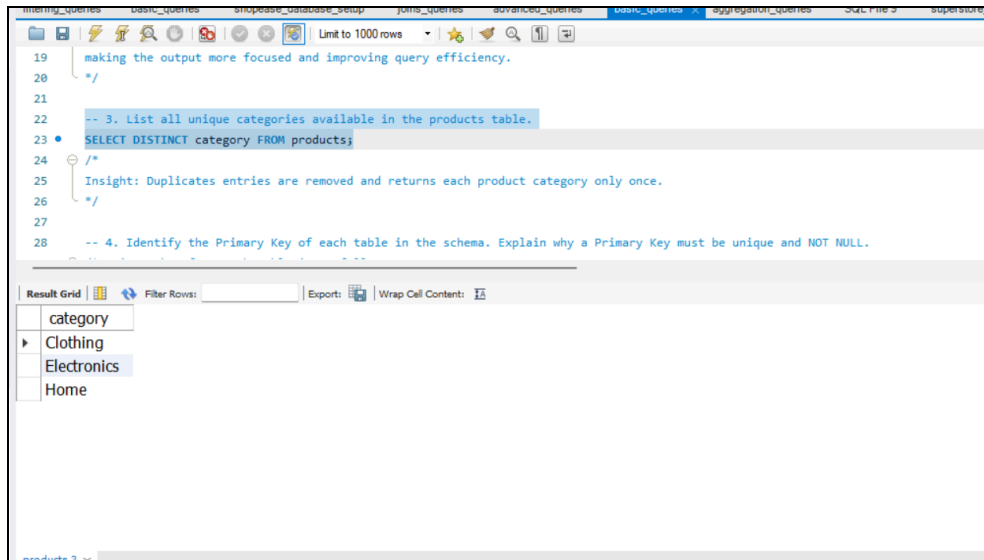
The screenshot shows a SQL IDE window with a script editor and a result grid. The script editor contains the following SQL code:

```
/*
Execute the selected portion of the script or everything, if there is no selection
*/
-- 2. Retrieve only the first_name, last_name, and city of all customers.
SELECT first_name, last_name, city FROM customers;
/*
Insight: Displays only the required customer details instead of all columns,
making the output more focused and improving query efficiency.
*/

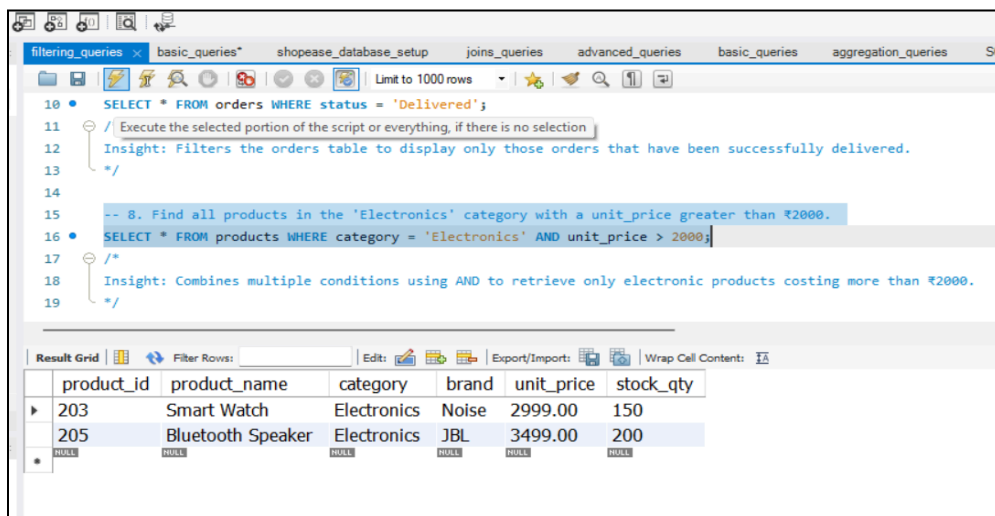
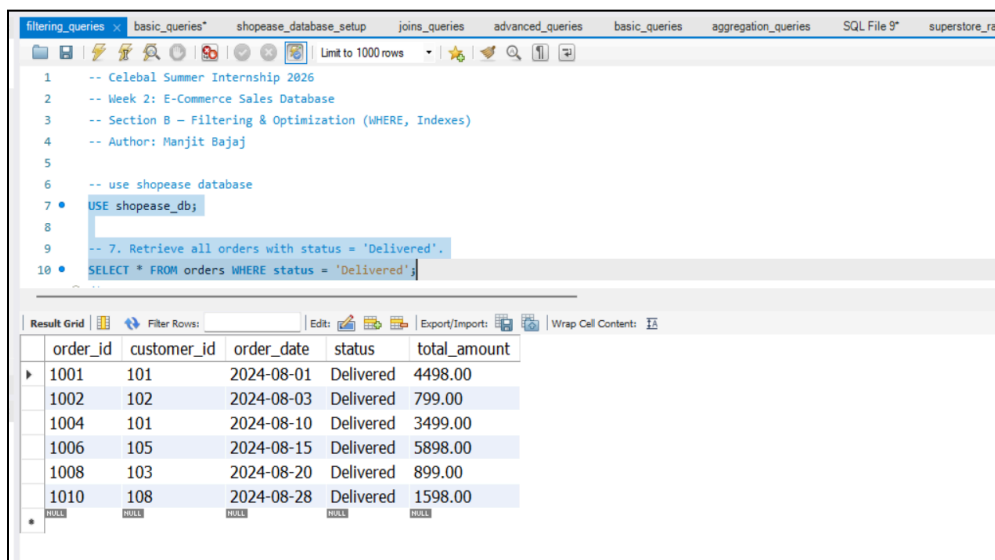
-- 3. List all unique categories available in the products table.
```

The result grid displays the output of the query, showing 3 columns: first_name, last_name, and city. The data is as follows:

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi



Section B — Filtering & Optimization (WHERE, Indexes)



filtering_queries x basic_queries* shopease_database_setup joins_queries advanced_queries basic_queries aggregation_queries SQL File 9* superstore_raw

Limit to 1000 rows

```

16 • SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;
17 /* Execute the selected portion of the script or everything, if there is no selection
18 Insight: Combines multiple conditions using AND to retrieve only electronic products costing more than ₹2000.
19 */
20
21 -- 9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.
22 • SELECT * FROM customers
23 WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31'
24 AND state = 'Maharashtra';
25 */

```

Result Grid | Filter Rows: | Edit: | Export/Imports: | Wrap Cell Contents: |

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

filtering_queries x basic_queries* shopease_database_setup joins_queries advanced_queries basic_queries aggregation_queries SQL File 9* superstore_raw

Limit to 1000 rows

```

28 /*
29 Execute the selected portion of the script or everything, if there is no selection
30 -- 10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.
31 • SELECT * FROM orders
32 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
33 AND NOT status = 'Cancelled';
34 */
35 Insight: Displays orders within the given dates and excludes cancelled orders.
36 */
37

```

Result Grid | Filter Rows: | Edit: | Export/Imports: | Wrap Cell Contents: |

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00

filtering_queries x basic_queries* shopease_database_setup joins_queries advanced_queries basic_queries aggregation_queries SQL File 9* superstore_raw

Limit to 1000 rows

```

62 • SELECT * FROM customers WHERE YEAR(join_date) = 2024;
63 /*
64 These query does not use the index. By applying the YEAR() function to the join_date column
65 it prevents MySQL from efficiently using an index on join_date.
66 Instead, the function must be evaluated for every row,
67 which can lead to a full table scan.
68 A SARGable (Search ARGument ABle) query compares the column
69 directly without applying functions to it.
70 */
71

```

Result Grid | Filter Rows: | Edit: | Export/Imports: | Wrap Cell Contents: |

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

The screenshot shows a SQL IDE window with the following tabs: filtering_queries, basic_queries*, shopease_database_setup, joins_queries, advanced_queries, and basic_queries. The query editor contains the following SQL code:

```
1 -- Celebal Summer Internship 2026
2 -- Week 2: E-Commerce Sales Database
3 -- Section C - Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
4 -- Author: Manjit Bajaj
5
6 -- Use the Shopease database
7 • USE shopease_db;
8
9 -- 13. Count the total number of orders in the orders table.
10 • SELECT COUNT(*) AS total_number FROM orders;
```

The result grid shows the following data:

total_number
12

The screenshot shows a SQL IDE window with the following tabs: filtering_queries, basic_queries*, shopease_database_setup, joins_queries, advanced_queries, basic_queries, and aggregation_queries. The query editor contains the following SQL code:

```
7 • USE shopease_db;
8
9 -- 13. Count the total number of orders in the orders table.
10 • SELECT COUNT(*) AS total_number FROM orders;
11 /*
12 Insight: Counts the total number of orders available in the orders table.
13 */
14
15 -- 14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.
16 • SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered';
```

The result grid shows the following data:

total_revenue
17191.00

The screenshot shows a SQL IDE window with the following tabs: filtering_queries, basic_queries*, shopease_database_setup, joins_queries, advanced_queries, basic_queries, and aggregation_queries. The query editor contains the following SQL code:

```
16 • SELECT SUM(total_amount) AS total_revenue FROM orders WHERE status = 'Delivered';
17 /*
18 Insight: Calculates the total revenue earned from all delivered orders only.
19 */
20
21 -- 15. Calculate the average unit_price of products in each category.
22 • SELECT Category, AVG(unit_price) AS average_unit_price FROM products GROUP BY category;
23 /*
24 Insight: Shows the average price of products for each category.
25 */
```

The result grid shows the following data:

Category	average_unit_price
Clothing	2699.000000
Electronics	2224.000000
Home	949.000000

filtering_queries basic_queries* shopease_database_setup joins_queries advanced_queries basic_queries aggregation_queries x SQL File

Limit to 1000 rows

```

22 • SELECT Category, AVG(unit_price) AS average_unit_price FROM products GROUP BY category;
23 /* Execute the selected portion of the script or everything, if there is no selection */
24 Insight: Shows the average price of products for each category.
25 */
26
27 -- 16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.
28 • SELECT status, COUNT(order_id) AS total_orders, SUM(total_amount) AS total_revenue
29 FROM orders GROUP BY status ORDER BY total_revenue DESC;
30 /*
31 Insight: Displays the number of orders and total revenue for each order status.

```

Result Grid Filter Rows: Export: Wrap Cell Content:

status	total_orders	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Pending	3	4495.00
Cancelled	1	2999.00

filtering_queries basic_queries* shopease_database_setup joins_queries advanced_queries basic_queries aggregation_queries x SQL File 9* supersto

Limit to 1000 rows

```

26
27 -- 16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.
28 • SELECT status, COUNT(order_id) AS total_orders, SUM(total_amount) AS total_revenue
29 FROM orders GROUP BY status ORDER BY total_revenue DESC;
30 /*
31 Insight: Displays the number of orders and total revenue for each order status.
32 The results are sorted from highest revenue to lowest.
33 */
34
35 -- 17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

```

Result Grid Filter Rows: Export: Wrap Cell Content:

category	Highest_price	Lowest_Price
Clothing	4599.00	799.00
Electronics	3499.00	899.00
Home	1299.00	599.00

filtering_queries basic_queries* shopease_database_setup joins_queries advanced_queries basic_queries aggregation_queries x SQL File 9* s

Limit to 1000 rows

```

34
35 -- 17. Find the most expensive (MAX) and cheapest (MIN) product in each category.
36 • SELECT category, max(unit_price) as Highest_price, MIN(unit_price) as Lowest_Price
37 FROM products GROUP BY category;
38 /*
39 Insight: Shows the highest and lowest product price in each category.
40 */
41
42 -- 18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)
43 • SELECT category , AVG(unit_price) as average_unit_price FROM products

```

Result Grid Filter Rows: Export: Wrap Cell Content:

category	average_unit_price
Clothing	2699.000000
Electronics	2224.000000

Section D - Joins & Relationships

The screenshot shows a SQL IDE window with the 'joins_queries' tab selected. The SQL script contains a comment for 'Section D - Joins & Relationships' and a query for problem 19. The query is an INNER JOIN between 'orders' and 'customers' tables, displaying order details along with customer names. The result grid shows 10 rows of data.

```
1 -- Celebal Summer Internship 2026
2 -- Execute the selected portion of the script or everything, if there is no selection
3 -- Section D - Joins & Relationships
4 -- Author: Manjit Bajaj
5
6 -- 19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date,
7 • SELECT o.order_id, o.order_date, c.first_name, c.last_name , o.total_amount
8 FROM orders o INNER JOIN customers c
9 ON o.customer_id = c.customer_id;
10 /*
```

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1011	2026-06-27	Priya	Patel	1598.00
1012	2026-06-27	Priya	Patel	1598.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5808.00

The screenshot shows a SQL IDE window with the 'joins_queries' tab selected. The SQL script contains a comment for problem 20 and a query using a LEFT JOIN between 'customers' and 'orders' tables. The result grid shows 10 rows of data, including customers with no orders (NULL values).

```
10 /*
11 Insight: returns only those orders that have matching customer
12 */
13
14 -- 20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for o
15 • SELECT * FROM customers c LEFT JOIN orders o
16 ON c.customer_id = o.customer_id;
17 /*
18 Insight: Left join returns every customer. If a customer has no orders
19 then order column contains NULL value.
```

customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1011	102
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1012	102
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105

The screenshot shows a SQL IDE window with the 'joins_queries' tab selected. The SQL script contains a comment for problem 21 and a query using JOINs across three tables: 'orders', 'order_items', and 'products'. The result grid shows 10 rows of data.

```
19 then order column contains NULL value.
20 */
21
22 -- 21. Write a query using JOINs across three tables (orders + order_items + products) to show: order_id, product_name, quantity, unit_price, and
23 • SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct
24 FROM orders o JOIN order_items oi
25 ON oi.order_id = o.order_id
26 JOIN products p
27 ON oi.product_id = p.product_id;
28
```

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1006	Wireless Earbuds	1	1499.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1005	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1006	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00

Section E - Advanced Concepts (CASE, ACID, Transactions)

The top screenshot shows a SQL IDE with the following query:

```
7 • USE shopease_db;
8
9 -- 24. Write a query using CASE to classify products into price tiers
10 • SELECT product_name, unit_price,
11 CASE
12 WHEN unit_price < 1000 THEN 'Budget'
13 WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-range'
14 ELSE 'Premium'
15 END AS price_tier
16 FROM products;
```

The result grid shows the following data:

product_name	unit_price	price_tier
Wireless Earbuds	1499.00	Mid-range
Cotton T-Shirt	799.00	Budget
Smart Watch	2999.00	Mid-range
Running Shoes	4599.00	Premium
Bluetooth Speaker	3499.00	Premium
Bedsheet Set	1299.00	Mid-range
Laptop Stand	899.00	Budget
Cushion Covers (Set)	599.00	Budget

The bottom screenshot shows a SQL IDE with the following query:

```
19 /*
20 Execute the selected portion of the script or everything, if there is no selection
21 -- 25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display t
22 • SELECT SUM( CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_order,
23 SUM( CASE WHEN status != 'Delivered' THEN 1 ELSE 0 END) as Not_Delivered_orders
24 FROM orders;
25 */
26 Insight: This query gives the count of delivered and not delivered orders.
27 We can use count() function as well.
28 /*
```

The result grid shows the following data:

Delivered_order	Not_Delivered_orders
6	6

Conclusion

This project demonstrates the practical application of SQL for analyzing a real-world retail sales dataset. Using sample shopease dataset, various SQL concepts such as data exploration, filtering, aggregation, grouping, sorting, conditional logic, and business-oriented analysis were implemented to extract meaningful insights.

Overall, this assignment strengthened the understanding of relational databases, SQL querying techniques, and analytical thinking, while providing hands-on experience in working with real-world business data.